

Fundamentele Algoritmilor

Principii de elaborare a unui program

1.1 Noțiuni introductive

Înțelegerea algoritmilor

Ce este un algoritm?

Definiție

Un algoritm este o secvență finită și bine definită de instrucțiuni, concepută pentru a rezolva o problemă specifică sau pentru a efectua un calcul.

Gândiți-vă la el ca la o **rețetă**: având ingrediente specifice (date de intrare), urmați pași exacți pentru a produce un preparat (ieșire) fără nicio ambiguitate.

Proprietăți Caracteristice

- ▶ **Finitudine**: Trebuie să se termine după un număr finit de pași.
- ▶ **Claritate**: Fiecare pas trebuie să fie riguros și fără ambiguitate.
- ▶ **Intrări/Ieșiri**: Are 0 sau mai multe intrări și produce minim o ieșire.
- ▶ **Eficacitate**: Operațiile trebuie să fie suficient de elementare pentru a fi realizabile pe un calculator.

Etapele rezolvării unei probleme



1. Analiza

Înțelegerea cerințelor problemei. Identificarea datelor inițiale (intrări) și a rezultatului dorit (ieșiri).



2. Proiectarea

Conceperea logicii. Implică crearea unei scheme logice sau scrierea în pseudocod pentru a mapa secvența pas cu pas.



3. Implementarea

Traducerea designului logic într-un limbaj de programare specific (precum Python, C++ sau Java) pentru a fi executat de mașină.

1.2 Date cu care lucrează algoritmi

Constante, Variabile și Expresii

Manipularea Datelor în Python

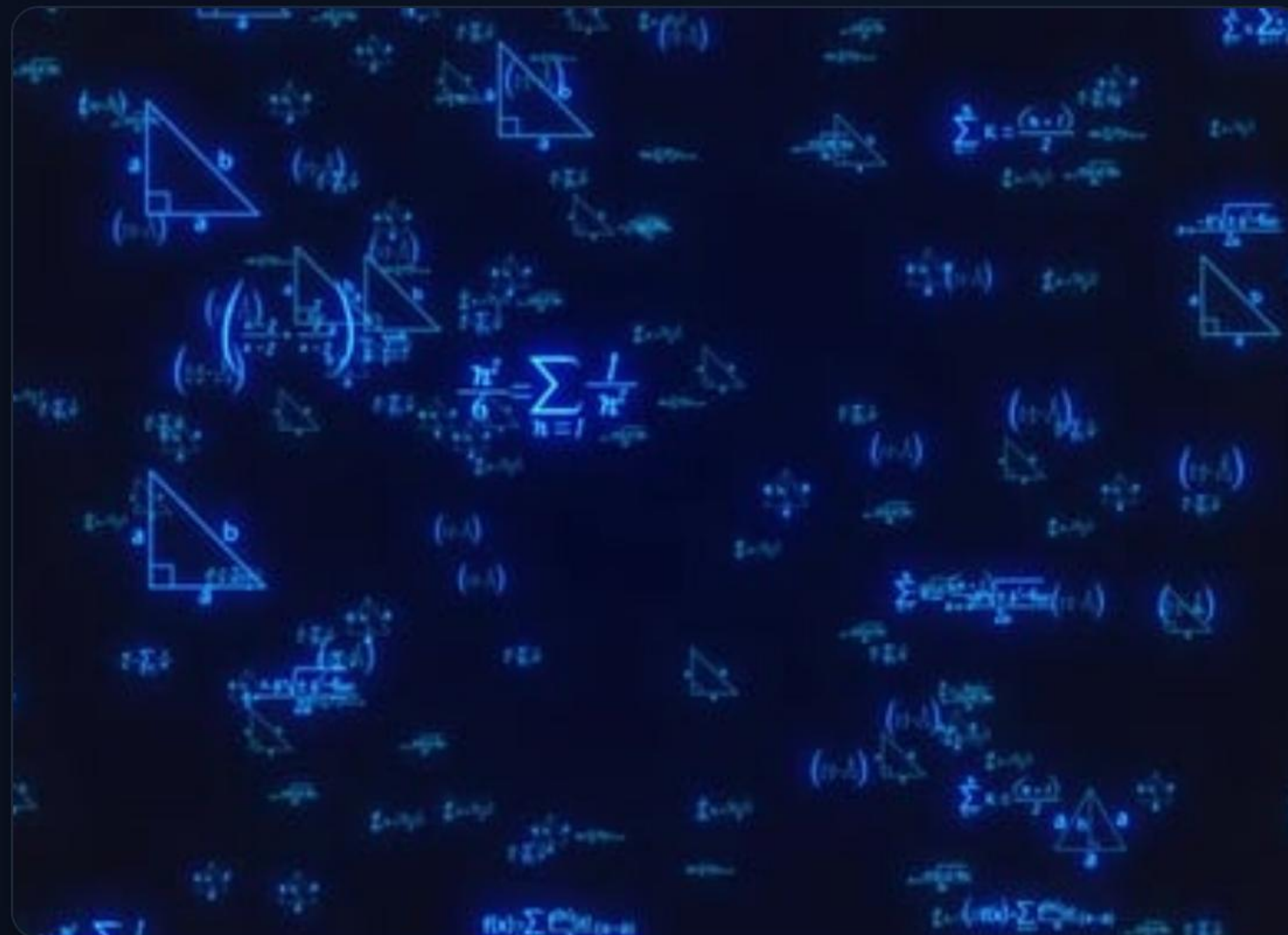
Algoritmii manipulează date. **Constantele** sunt valori care nu se schimbă în timpul execuției, în timp ce **Variabilele** sunt containere pentru valori. Le combinăm folosind **Expresii**.

```
# Constantă (convenție: litere mari)
PI = 3.14159

# Variabilă (poate fi suprascrisă)
raza = 5

# Expresie (combină operatori și date)
aria = PI * (raza ** 2)

# Afișarea rezultatului
print("Aria cercului:", aria)
```



1.3 Reprezentarea algoritmilor

Structuri de control și flux de execuție

Cele trei structuri fundamentale



Secvențială

Comportamentul implicit.
Instrucțiunile se execută liniar,
una după alta, de sus în jos,
fără nicio omitere.



Alternativă

Luarea deciziilor. Algoritmul
verifică o condiție și alege o
cale de execuție pe baza
rezultatului Adevărat sau Fals.



Repetitivă

Buclele. Un bloc specific de
instrucțiuni este repetat atâta
timp cât o condiție logică
rămâne Adevărată.

Structura Alternativă (if/else)

Ramificarea permite programelor noastre să fie dinamice. Codul evaluează o condiție logică și alege o rută specifică de execuție.

```
# Date de intrare
nota = float(input("Introdu nota: "))

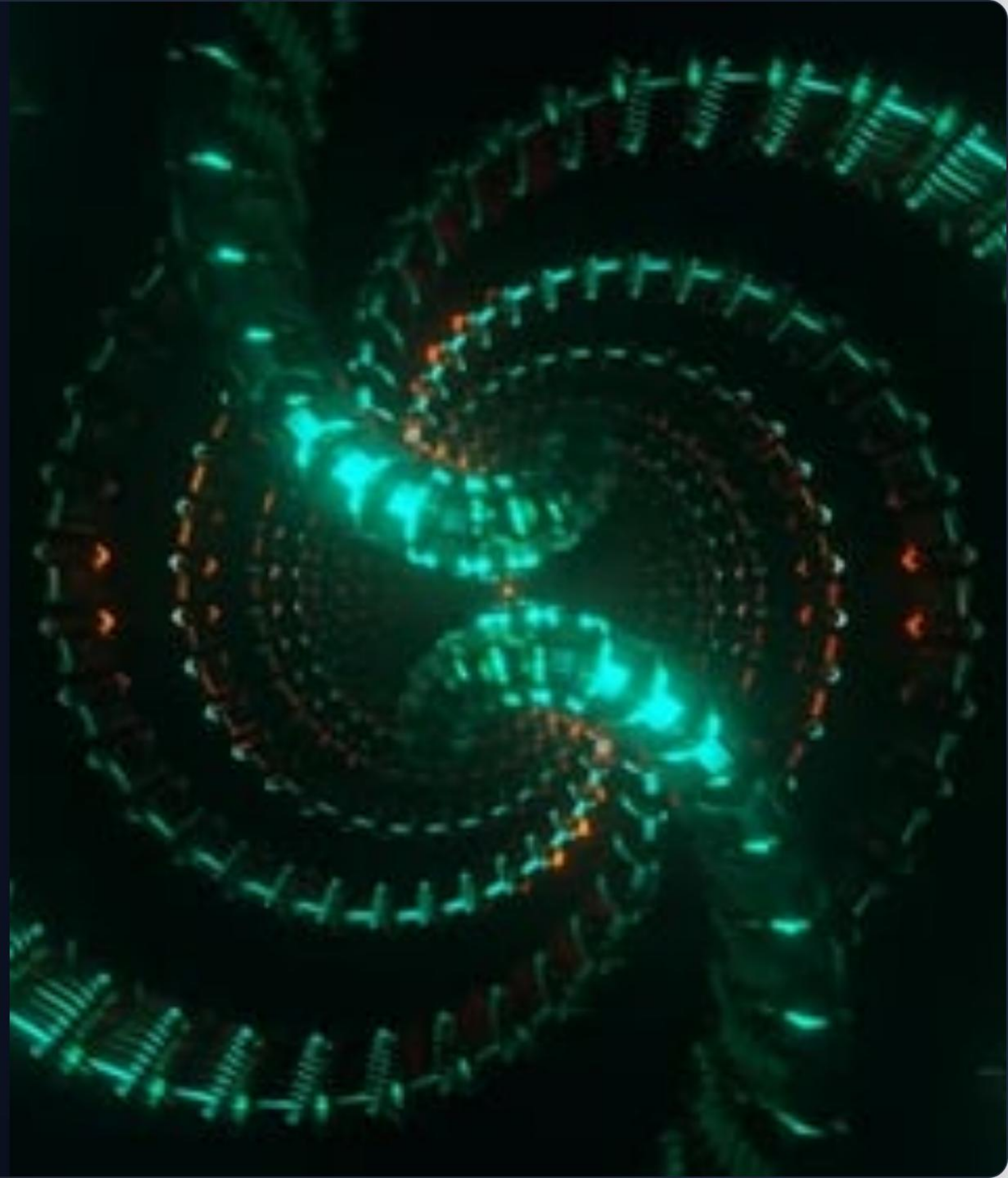
# Structură alternativă de decizie
if nota ≥ 5.0:
    print("Status: Examen promovat!")
    print("Felicitări.")
else:
    print("Status: Picat.")
    print("Trebuie să repeți examenul.")
```



Structura Repetitivă (Bucle)

Structurile repetitive (buclele) elimină redundanța codului executând aceeași secvență de instrucțiuni de mai multe ori.

```
# Calcularea unui factorial folosind un for  
target = 5  
total = 1  
  
for i in range(1, target + 1):  
    total = total * i  
    print("Pasul", i, "→", total)  
  
print("\nFactorial final:", total)
```



Evaluare

Sfârșitul capitolului: Fundamentele elaborării algoritmilor

Vă rugăm să completați fișa de autoevaluare (S1 - S7).

Image Sources



https://elements-resized.envatousercontent.com/elements-video-cover-images/0084181b-f79a-409e-9941-4f8fc8c2ba2b/video_preview/video_preview_0002.jpg?w=500&cf_fit=cover&q=85&format=auto&s=1c0f44b49b57b18bd52137ebcc3a4071b069162147e224f157485594731cc4d5

Source: elements.envato.com



https://images.pond5.com/neon-arrows-diverging-choices-path-footage-308708387_icon1.jpeg

Source: www.pond5.com



https://elements-resized.envatousercontent.com/elements-video-cover-images/41e970c9-f1a0-4faf-a89a-a64719d4d9fb/video_preview/video_preview_0000.jpg?w=500&cf_fit=cover&q=85&format=auto&s=3f2ecf546f8f202ac2431ad983a26da3185c3f742d69b286ecc3b2305972fab0

Source: elements.envato.com